

End-of-Studies Internship Report

Full-stack feature development on an occupational-health SaaS platform

Padoa — Paris

Author:
BOUNOUA Ilyas

Ecole Centrale de Nantes — Information Systems

Six-month engineering internship, April – September 2026

Portfolio version — public

This document is a redacted edition of an internship report written for Padoa, an occupational-health software company. Internal repository, module, database and configuration-switch names, ticket identifiers, internal documentation references, commercial figures, client names, screenshots of internal tooling and the names of colleagues and supervisors have all been removed. What remains is the engineering reasoning: the problems, the design arbitrations, the trade-offs and their consequences. Technical objects are therefore described by their function rather than named.

Contents

I	Introduction & Onboarding	2
I.1	Technical Onboarding and Development Environment	2
I.2	The Frontend Ecosystem and Environments	2
I.3	Leveraging AI for Technical Growth and Analysis	2
I.3.1	Codebase Exploration and Learning	3
I.3.2	Support for the Analysis Phase	3
I.3.3	Professionalism and Best Practices	3
II	Macro-Context & Market Analysis	4
II.1	The French Occupational Health Framework (SPSTI / SPSTA)	4
II.2	Market Position and Expansion Strategy	4
II.3	Internal Organization and Position of my Team	4
III	Agile Methodology & Engineering Operations	6
III.1	Agile Rituals and Sprint Cycles	6
III.2	Operational Rotations	6
III.2.1	The On-Call Support Role	6
III.2.2	The Iteration-Based Scrum Master	6
III.3	Technical Slicing Framework	7
IV	Internship Objectives & Global Roadmap	8
IV.1	The Mandate: Subject Framing	8
IV.2	Initial Milestones and Timeline	8
IV.3	High-Level Functional Expectations	8
V	Case Study: Cross-Entity Duplicate Identification through Shared Company Identifiers	9
V.1	Phase 1: The existing system and its limits	9
V.2	Phase 2: Design decisions	9
V.3	Phase 3: What I implemented	9
V.4	Phase 4: Validation and critical reflection	10
VI	Case Study: Resolving and Publishing a Sector's Social Worker on Two Portals	12
VI.1	Phase 1: The existing system and its limits	12
VI.2	Phase 2: Design decisions	12
VI.3	Phase 3: What I implemented	13
VI.4	Phase 4: Validation and critical reflection	13
VII	Case Study: Verifying and Enriching Automatically Created Prospects	14
VII.1	Phase 1: The existing system and its limits	14
VII.2	Phase 2: Design decisions	14
VII.3	Phase 3: What I implemented	14
VII.4	Phase 4: Validation and critical reflection	15
VIII	Case Study: Bulk Revocation of a User's Access Across Every Member Company	17
VIII.1	Phase 1: The existing system and its limits	17
VIII.2	Phase 2: Design decisions	17
VIII.3	Phase 3: What I implemented	18
VIII.4	Phase 4: Validation and critical reflection	18
IX	Conclusion and Personal Assessment	21
IX.1	Professional Summary & Collaborative Outcomes	21
IX.2	Personal Development & Engineering Posture	21
X	Glossary	23

I Introduction & Onboarding

This report details the end-of-studies internship I carried out from April to September 2026 at Padoa, a French company publishing a SaaS platform for occupational health and prevention services, located in Paris. As the final requirement for the engineering degree at Ecole Centrale de Nantes, specializing in Information Systems, this six-month mission represents the transition from academic learning to professional engineering practice. The internship focused on full-stack feature development inside the product pole responsible for the lifecycle of member companies and prospects.

The primary objectives of this mission were to design and implement production-ready full-stack components, optimize the pipelines that keep the backend search index in sync, and dismantle legacy technical debt limiting the platform's scale.

Note on vocabulary : Two terms recur throughout this report and are each used in one sense only. A **member company** (*adhérent* in the product's own French vocabulary) is a company that has joined an occupational health service and whose employees that service monitors. A **prospect** is a company that has not joined yet. The service may identify and create the record itself, whether commercially or automatically from an administrative declaration, or the company may create it itself by completing an eligibility questionnaire. The word *lead* is deliberately not used as a synonym, since inside Padoa it belongs to the German commercial vocabulary rather than to the French one.

I.1 Technical Onboarding and Development Environment

The initial phase of the internship was dedicated to a rigorous technical onboarding designed to align my development environment with Padoa's engineering standards. This process was facilitated by a bootstrap repository serving as the central entry point for all developers, from which the tooling is installed and the secondary repositories are cloned.

Key steps in this setup included:

- **IDE and Development Tools:** Adopting an AI-assisted IDE as the primary editor, with SSO licensing, and installing the inspection tools the team relies on outside the editor: the Redux DevTools browser extension to follow client state, and an ElasticSearch browser client to read the search indices directly.
- **Connectivity:** Setting up the corporate VPN and the internal browser extension used to navigate between client environments and internal tools.
- **Architecture Overview:** Attending the internal engineering academy sessions, which provided deep dives into the frontend and backend architectures. These presentations clarified how the various Yarn workspaces are structured and how the shared core library supports the main applications.

In practice, this setup meant that every feature was validated across three layers from a single workstation: the IDE for source and diffs, the application itself served locally (the front-end against a locally running backend server and its database, both started from the bootstrap scripts), and the Redux DevTools and ElasticSearch browser panels for client state and search-index inspection. That three-layer loop is reused later when verifying the duplicate counts of Section V.

I.2 The Frontend Ecosystem and Environments

A significant portion of my technical immersion involved understanding the platform's specific frontend patterns. The application relies heavily on a normalized **Redux store** to maintain data consistency across complex data structures. The frontend components are integrated in a modular Angular framework, using Yarn workspaces to separate concerns. I also became familiar with the environment strategy used for testing and validation:

- A primary **staging environment** running pseudonymized data and the latest code from the master branch.
- A **production-like environment**, also on pseudonymized data, used to replicate and debug production issues safely.

I.3 Leveraging AI for Technical Growth and Analysis

In alignment with Padoa's modern engineering culture, I integrated an AI-powered IDE into my daily workflow. My use of this tool was guided by a pedagogical approach, focusing on accelerating my understanding

of the codebase and refining my technical decision-making rather than simple code generation.

I.3.1 Codebase Exploration and Learning

Navigating a large-scale monorepo with specific architectural patterns (such as normalized Redux stores) presents a significant learning curve. I used the IDE's context-aware capabilities to deconstruct complex logic. When encountering unfamiliar internal libraries or legacy components, I used the AI to explain the data flow and the rationale behind specific implementations. This reduced the time spent on code discovery and allowed me to focus on high-level architecture. By asking the AI to suggest best-practice implementations for RxJS streams or TypeScript interfaces, I was able to learn the team's specific coding standards more rapidly, effectively using the tool as a 24/7 technical mentor.

I.3.2 Support for the Analysis Phase

The AI proved particularly valuable during the **technical analysis and weighting** ritual described in Section III. Before presenting a ticket to the sub-squad, I used the tool to identify edge cases. I would prompt the IDE to analyze my proposed solution against existing service logic to find potential side effects. This led to more robust technical analyses and more accurate weighting of tickets. It assisted in articulating my technical choices, helping me prepare for the team's feedback during analysis sessions by anticipating potential performance or maintainability issues.

I.3.3 Professionalism and Best Practices

Consistent with Padoa's onboarding guidelines, the use of AI was handled with professional rigor. All AI-generated suggestions were manually reviewed, tested, and adapted to ensure they met the team's Definition of Done and linting standards. This ensured that while the tool assisted in building my capacities, the ultimate responsibility for the code and its logic remained strictly with me as the engineer. This collaborative interaction between the developer and the AI served as a powerful catalyst for my transition from a student perspective to an active engineering role within the squad.

II Macro-Context & Market Analysis

To understand Padoa's role, we must examine the socio-economic scope of the French occupational health and prevention ecosystem. This market is highly regulated and undergoes continuous digital transformation.

II.1 The French Occupational Health Framework (SPSTI / SPSTA)

The French occupational health system is split into two primary operational models:

- **SPSTI (Services de Prévention et de Santé au Travail Inter-entreprises)** : These are mutualized services that monitor employees from multiple small-to-medium enterprises. In France, SPSTIs monitor over 16 million employees with an aggregate national budget of €1.5 billion [1, 4].
- **SPSTA (Services de Prévention et de Santé au Travail Autonomes)** : These are internal health services dedicated exclusively to a single large enterprise (usually over several thousand employees).

This split is the product of a long institutional history: French occupational medicine was built after the 1946 law around employer-funded services whose actors, missions and monitoring indicators were progressively codified over the following decades [3]. That legacy explains both the fragmentation of the sector and the heterogeneity of its information systems, which the platform has to accommodate.

Historically, this sector has suffered from severe systemic challenges [2]:

1. **Chronic shortage of occupational physicians** : The declining number of certified occupational doctors makes it impossible to conduct mandatory face-to-face medical checkups for all employees within traditional timeframes.
2. **High absenteeism and mental health risks** : Rising worker burnout, stress, and physical health concerns require shifts from curative medicine to proactive, preventive risk assessments.
3. **Severe technical debt** : Most occupational health services traditionally relied on fragmented paper records or legacy, desktop-based software. This prevented collaboration between doctors, employers, and employee portals.

II.2 Market Position and Expansion Strategy

Padoa was founded in 2016 to address these systemic gaps by building a modern, cloud-native SaaS platform. By automating administrative workflows (scheduling, billing, document ingestion) and offering collaborative interfaces for doctors, companies, and employees, it has captured a leading position on the French market, with several million employee records monitored across its client services.

As part of its growth strategy, Padoa is expanding along two strategic vectors:

1. **Capturing autonomous accounts** : Moving from an exclusive focus on mutualized inter-enterprise services (SPSTI) toward large, autonomous corporate services (SPSTA), which require dedicated internal configuration systems. Section VIII shows the concrete form this takes in the product: the wording of a screen is resolved against the type of customer, so the same component names the same entity differently depending on whether the client is a mutualized service or an autonomous one, rather than being duplicated.
2. **Internationalization** : Capturing markets outside France, Germany in particular. There is no separate German edition of the platform: the large majority of the product remains shared. The real distance between the two markets is not a list of technical options but a way of thinking about the customer. A French SPSTI is a mutualized non-profit body and reasons in terms of member companies that join it and are represented on its board; the German market approaches the same activity as a business selling a service to clients. That difference shapes vocabulary and expectations far more than it forks the code.

II.3 Internal Organization and Position of my Team

Padoa's internal structure directly reflects the two strategic vectors described above. It is functionally split into directorates covering client operations, product, engineering, human resources and revenue, with the product directorate itself divided between the historical core business and the newer units created to serve autonomous services and foreign markets.

Within the technical and product directorates, engineering is organized in **squads** aligned with the product poles, each squad being subdivided into **sub-squads** of four to six people combining developers, a Product Manager and a designer. My internship took place in the sub-squad owning the lifecycle of member companies and prospects, which is precisely the functional perimeter of the four case studies presented in Sections IV and beyond.

This organization has two practical consequences on the way I worked. First, a sub-squad owns its perimeter end-to-end (database migrations, backend services, shared npm packages and Angular front-ends), so every feature I delivered was full-stack by construction. Second, because several sub-squads share the same monorepo and the same shared packages, any structural change (a new contact type, a new search-index field) has to be discussed with the neighbouring teams, and frequently challenged by them, before being merged.

III Agile Methodology & Engineering Operations

Operating as an engineer in a large-scale tech company requires structured methodologies to maintain high code quality and deliver business value predictably.

III.1 Agile Rituals and Sprint Cycles

The sub-squad follows a structured two-week sprint cycle. The team coordinates through established rituals to ensure transparency and manage project delivery:

- **Daily stand-up (15 min)** : Every morning, the team synchronizes on the previous day's progress, the plan for the current day, and any blockers.
- **Analysis and weighting (twice a week)** : Team members present a detailed technical analysis of their assigned tickets. The team critiques the proposed implementation, discusses potential edge cases, and collectively weights the ticket on a Fibonacci scale based on technical complexity. Each ticket carries both the weight agreed before development and the final weight recorded afterwards, which makes the estimation error of the sub-squad measurable rather than anecdotal.
- **Sprint retrospective** : At the end of each two-week cycle, the team reviews what went well and what could be optimized, establishing concrete action items for the next sprint.
- **Weekly one-on-ones** : Individual checkpoints with my manager to discuss professional development, alignment, and internship progress.

The sprint itself is tracked on a board counting the tickets of the running iteration by workflow status, from ready-to-analyze through development, test and review, and finally merged. Above the sprint horizon, the sub-squad also tracks its multi-sprint projects on a dedicated page measuring the advance or delay of each project in person-days and recording a fixed owner for each one. That owner stays on the project for its duration, except when they are unavailable and the work must still move, in which case another member temporarily takes over.

III.2 Operational Rotations

To maintain production stability and team alignment, two operational roles rotate among the engineers:

III.2.1 The On-Call Support Role

The on-call support role is assigned to one engineer on a weekly rotation. During this week, the engineer is isolated from standard sprint feature tickets to focus entirely on maintaining application health, handling production incidents, and performing manual validation sweeps on the master-aligned staging target.

A major incident resolved during my rotation involved an asynchronous synchronization failure between the status of a member company and the lifecycle of its employees' contracts. When a member company was automatically flagged as restricted for unpaid balances by the billing dunning workflow, its status correctly changed to terminated. However, the system failed to trigger the cascade closure that should follow, leaving individual employee contracts active. To resolve this, I refactored the backend status modification hooks to trigger the identical cascade closure path as manual administrative actions. Additionally, because production data had already diverged, I engineered an SQL data mitigation script, validated through a notebook environment against anonymized staging structures before running the retroactive backfill query on production.

Incoming production issues reach the squad as support tickets, automatically mirrored into the sub-squad's triage table. Each entry carries its origin, its internal status and its priority, and part of the role consists in claiming the unassigned tickets and routing those that belong to another squad. Tickets arrive in several languages, which follows from the internationalization described in Section II: the platform serves clients in several countries, even though most of its features remain shared.

III.2.2 The Iteration-Based Scrum Master

The Scrum Master role rotates on a per-sprint basis (every two weeks). The Scrum Master directs the daily stand-ups, keeps the sprint board faithful to the real state of the work, carries the sub-squad's dependencies towards the other teams, and leads the retrospective. Holding the role moved my attention from my own tickets to the flow of the whole sub-squad, which is the substance of what it teaches.

III.3 Technical Slicing Framework

To deliver features reliably, the sub-squad implements a technical slicing framework:

1. **Kick-off dev phase** : Product Managers, designers, and engineers review the mockups together, identifying UI and UX ambiguities and anticipating boundary flaws in the data model.
2. **Slicing blueprint** : The requirements are decomposed into a dependency map of concrete technical tasks (database migrations, backend service hooks, package bumps, frontend components).
3. **Technical analysis** : The engineer maps the data path from the Angular client-side state stores down to the database tables, writing a technical plan *before* writing any code.
4. **MVP and feature flags** : Full-stack implementations are hidden behind temporary feature flags. This allows the team to merge functional code early and exercise it directly through the API, with an HTTP client, before any interface is in place.

The complete engineering delivery cycle is structured around a strict Definition of Done, illustrated in the flowchart of Figure 1.

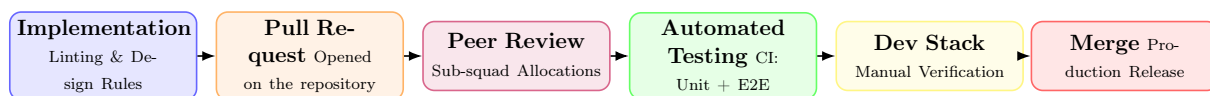


Figure 1: The Definition of Done lifecycle followed by every delivered change

IV Internship Objectives & Global Roadmap

This section details the formal boundaries of my six-month internship, separating the engineering mandate assigned to me from the technical implementation cases described in the subsequent sections of this report.

IV.1 The Mandate: Subject Framing

My presence in the sub-squad was governed by a primary mandate: to remove operational friction from the lifecycle of prospects and member companies. Rather than simply developing isolated user interfaces, I was assigned to:

1. Surface, on the prospect list, the duplicates that share a company identifier, by pre-calculating the counts through an event-driven pipeline instead of computing them at display time.
2. Modernize legacy contact configurations on the employer and employee portals, so that occupational health services can expose their own team structures rather than a fixed list.
3. Verify and complete the prospects created automatically from administrative pre-employment declarations, using an external legal-information provider without exceeding its request budget.
4. Replace a manual database procedure still executed by developers on behalf of support with a self-service bulk action, guarded against the inconsistent states that procedure could silently produce.

IV.2 Initial Milestones and Timeline

The six-month timeline was structured around three key milestones to align codebase learning with technical delivery:

- **Milestone 1 (months 1–2) — Onboarding and asynchronous synchronization :** Setting up the development environment, understanding the Redux data flows, and implementing the event listener hooks for duplicate tracking in the search index.
- **Milestone 2 (months 3–4) — Contact form refactoring and social worker resolution :** Refactoring the contact configuration form, writing the query that resolves the titular social worker of a sector from its medical team, and deploying the resulting contact type on both portals behind feature flags.
- **Milestone 3 (months 5–6) — Prospect enrichment, bulk operations and collective verification :** Connecting an external legal-information API to the pre-employment declaration pipeline, adding the cache that protects its request budget, moving a manual support procedure into a background bulk action arbitrated by two feature flags, and validating the pipelines through collective testing sessions and SQL auditing.

IV.3 High-Level Functional Expectations

All engineering deliverables were constrained by strict functional expectations defined by Padoa's growth strategy:

- **Performance :** Search and listing views must stay responsive whatever the size of the catalogue. They are served by the Elasticsearch index rather than by the relational database, which means anything they display has to be computable when the data is indexed rather than when the page is read.
- **Strategic scale :** The code must support large autonomous corporate accounts and remain ready for market-specific features, in particular those destined for Germany, without forking the shared core.
- **Security and compliance :** Since the platform processes sensitive data, every change must strictly comply with the General Data Protection Regulation (RGPD), the French health-data hosting certification (HDS), and the national health identifier (INS) standards.

V Case Study: Cross-Entity Duplicate Identification through Shared Company Identifiers

This case study details the full-stack development of a system that identifies duplicate company records by matching them on their SIREN. A SIREN is the 9-digit identifier assigned by INSEE to every French company; a SIRET is the 14-digit identifier of one of its establishments, formed by that SIREN plus a 5-digit establishment number. Two records that share a SIREN are therefore the same legal company. That equivalence holds even though the two records may live in different tables, one among member companies and the other among prospects.

What existed, and what I added : The platform already stored prospects and member companies in two separate tables, and already served their lists from the Elasticsearch search index rather than from SQL, as it does for every list view. What it did not do was relate two records describing the same legal company: an operator reading the prospect list had no way of knowing that the company in front of them was already known elsewhere in the platform. I added that relation, in the form of two counts carried by each indexed prospect document, the event handling that keeps those counts correct when the underlying data changes, and the three interface elements that expose them: a list filter, a row indicator and a modal listing the duplicates.

As on every feature I delivered during the internship, the work was shipped behind a feature flag: a switch that keeps unfinished behaviour out of reach of every client until the team chooses to release the feature in production for everyone.

V.1 Phase 1: The existing system and its limits

During the beta-test of a previous feature, operational users reported that they could not tell, from the prospect list, whether a prospect already existed as a member company or as another prospect. Without that visibility they wasted time qualifying companies that were already in the database. The product team therefore asked for the duplicate information to appear on the prospect list itself, together with a filter that isolates rows that have duplicates.

V.2 Phase 2: Design decisions

The engineering question was where the two counts get computed, not which engine serves the list: like every list view of the platform, this one is already served by the search index rather than by a SQL query. Counting the duplicates of each row at the moment a page is displayed, whether by joining back to the relational database or by aggregating over the whole index on every page, makes the cost of showing that page grow with the size of the catalogue.

We therefore compute the two counts (how many prospects and how many member companies share the identifier) when the data changes rather than when it is read, and store them on the Elasticsearch document itself. Displaying a page then costs nothing more than reading two integers that are already there. The computation lives in a dedicated indexing strategy: a handler that rebuilds those counts in the background whenever a relevant event arrives. Concretely, the strategy's fetch query assembles two conditional aggregates, one over the prospect statuses and one over the member-company statuses, into a single structured column, which is mirrored by a strictly typed object in the index mapping exposing the two integer sub-fields read at query time.

V.3 Phase 3: What I implemented

To keep the counts accurate I wired that strategy into the event cascade. Counts are recomputed asynchronously when:

- **Establishment updates :** If a user edits an establishment's SIRET and the SIREN part actually changes, the company leaves one cluster and enters another. Only then does the engine reindex both the source and the target clusters. An edit that leaves the SIREN unchanged does not move the company and does not trigger that cascade.
- **Status mutations :** When a prospect becomes an active member, the event bus asks every sibling document sharing that identifier to refresh its prospect and member counts.

On the frontend, a filter on the prospect list isolates rows that have at least one duplicate, and an info icon appears on those rows. Hovering it shows the breakdown (how many prospects, how many member companies), with the wording following the count through internationalized pluralization. Clicking the action button opens a modal that lists every prospect and member company sharing the identifier, with their detailed information. The data flow of this asynchronous reindexing pipeline is illustrated in Figure 2.

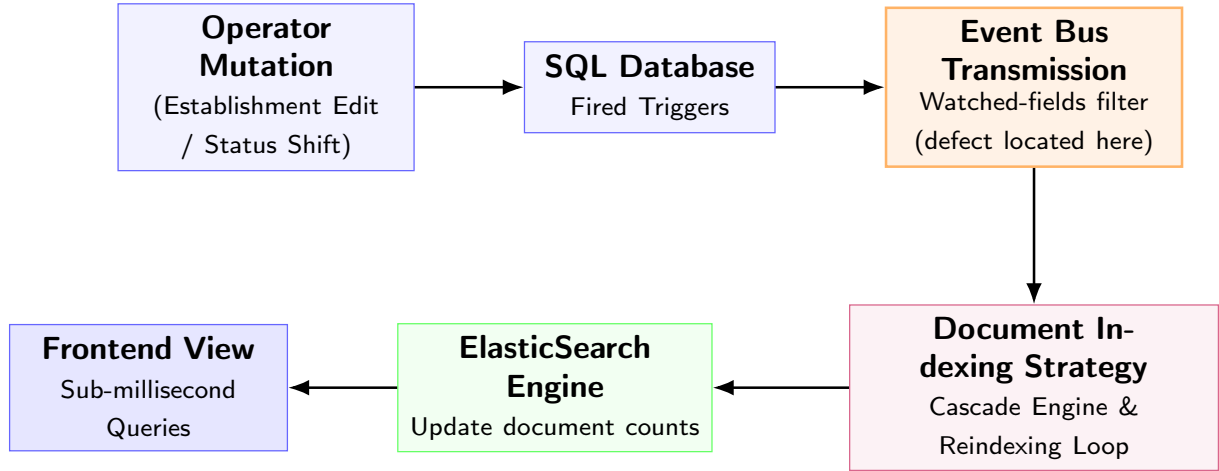


Figure 2: Asynchronous event reindexing data-flow pipeline. The watched-fields filter of the event bus is the stage where the synchronization defect described in Phase 4 was located

V.4 Phase 4: Validation and critical reflection

Because the displayed counts are the end of a three-layer chain (SQL mutation, event bus, ElasticSearch document), validating the feature required inspecting all three layers simultaneously rather than trusting the interface alone. My verification loop consisted in triggering a mutation from the CRM, following the resulting actions in the Redux DevTools action log, and confirming in the ElasticSearch browser client that the corresponding prospect index had been re-indexed with the expected count scalars.

Beyond this individual loop, the sub-squad conducted a collective peer-testing session across different runtime environments. During this session, we discovered two bugs:

- **Event bus synchronization gap** : Member-company history is modelled by writing a date of end on the principal establishment, not by deleting or closing the row. When that date was set, the duplicate counts on sibling prospects stayed stale, because the date-of-end field was not among the fields the event bus was watching on the establishment table. I registered the field on that trigger, so that it belongs to the payload the trigger may transmit and appears among the matched watched fields, and taught the indexing strategy to read both the old and the new identifier from an update, so that writing a date of end now reindexes the peers.
- **Modal rendering failures** : The duplicate modal crashed when opened on a prospect that had no associated member-company identifier, because the client assumed that identifier was always present. I made it optional. A second defect in the helper that formats the list returned nothing when the indexed document count was missing; it now treats a missing count as zero and still shows the prospects.

Both fixes are covered by regression tests on the indexing strategy (including the case where a date of end is written) and on the duplicate search filter.

The exact scope of the cascade was itself clarified during peer review. A reviewer asked why the member company whose date of end has just been set is not in the list of documents to reindex. It is excluded implicitly, because the resolver that collects peers only keeps records still considered active prospects, that is those whose date of end is still empty. Writing that reasoning down in the review thread is what convinced the reviewer, and it is a good illustration of a recurring lesson: an implicit invariant that nobody can restate on demand is indistinguishable from a bug.

Critical reflection : Pre-calculating the counts does make the cost of displaying a page independent of the size of the catalogue, and the volumes involved are modest anyway: the number of records that actually share one identifier is small, and even a cluster of a thousand documents would be cheap for ElasticSearch to

serve. So the design is not vindicated by the performance argument alone, and its real limits sit elsewhere. The first is the cost of the indexing calculation itself: if that computation is slow, reindexing a single document is enough to add latency or a CPU spike on the pipeline. The second is the interface. When an operator changes a status from the list, the page can refresh before the background reindex has finished, so the row the user is looking at can be visually out of date relative to the data they have just written. That race, not the size of the cluster, is the consistency problem this design has to live with.

VI Case Study: Resolving and Publishing a Sector's Social Worker on Two Portals

The domain, in four sentences : An occupational health service divides its territory into *sectors* and attaches to each sector a *medical team*: an occupational physician, nurses, and possibly specialists, among whom the social workers who support employees in difficulty. Every member company belongs to a sector, so every employee inherits the medical team of their company's sector. Both the employer portal, used by the administrators of a company, and the employee portal, used by the employees themselves, display a card listing the contacts drawn from that team. Which contacts appear on that card is configured by the service, per company status, in the administration panel, by choosing a *contact type* for each line.

What existed, and what I added : Contact types came in two kinds. Some are fixed, meaning an administrator types a name and a phone number by hand; others are resolved by the platform, which looks the person up in the data. Physicians and nurses were already resolved automatically. Social workers were not, so a service wanting to publish one had to enter it by hand on every company file, which is not realistic for a service with thousands of member companies, and the entry went stale as soon as the assignment changed. I added a contact type for the social worker that resolves by itself, from the company to its sector and from the sector to the social worker holding the titular assignment on its team, and made it available on both portals. Along the way I had to lift a constraint in the administration form that prevented this kind of contact from publishing a phone number at all.

The work was isolated behind two feature flags: one hiding the new contact type while it was being built, and one governing the employee-portal card, which was opened service by service.

VI.1 Phase 1: The existing system and its limits

The friction had two independent halves, which is why the subject was cut into several tickets.

The first is the manual entry described above: the platform had no way of naming the social worker of a sector by itself, so the information had to be copied onto each company file and maintained there by hand.

The second is a constraint in the administration form, and it is the reason the manual workaround was not usable either. Each contact line carries a set of checkboxes deciding which pieces of information are actually published to the portals: the name, the e-mail, the phone number. Those checkboxes were displayed only for contacts attached to a physical location, and hidden for contacts attached to a sector. A social worker belongs to a sector, so no phone number could be published for one, even when entered by hand. Since publishing a telephone number is the entire point of the card, the form had to be corrected before the new contact type could be of any use.

VI.2 Phase 2: Design decisions

Resolving rather than storing. The assignment already exists in the platform, as part of the medical team of a sector. Copying it onto every company profile would duplicate thousands of times a fact that is written once, and every copy could then fall out of date. The contact is therefore resolved when the card is read, by walking from the company to its sector and then to its team.

What counts as the social worker of a sector. A team member is kept only if two conditions hold: their profile is classified as a social worker, and their assignment to that sector is the titular one rather than a replacement.

What to do when the data is ambiguous. Nothing prevents a service from attaching several titular social workers to the same sector, which is a configuration mistake rather than an intended state. The first implementation asked for the whole list and displayed a contact only when that list held exactly one entry, so a service that had made this mistake saw no contact at all and nothing explaining why. We chose the opposite trade-off: the query asks for a single row and the card shows the first social worker it finds. A slightly arbitrary contact is more useful to an employee in difficulty than a silently empty card.

VI.3 Phase 3: What I implemented

On the backend, the resolution is a dedicated query written as a Common Table Expression, that is a named sub-query the existing contact machinery can consume like any other source. It joins the member company to its sector, then the sector to its medical team, and keeps the member whose assignment is marked titular and whose profile carries the social-worker scope. I also relaxed the pre-existing sector contact query, which required both an e-mail and a phone number to return anything, so that a phone number alone is enough.

That last change is where a PostgreSQL type mismatch stopped me for a while. The scope of a team member is stored as an array of strings, and I first wrote the filter as an array-contains comparison (`@> ARRAY[...]`), which the database rejected because of the exact type of the column; an element-of-array test (`= ANY(...)`) matches how the column is stored and resolved it. The lesson was cheap but real: raw SQL inside a typed codebase gets no help from the compiler, so only running the query tells you whether it is correct.

On the administration form, the constraint described in Phase 1 was a predicate listing the two location contact types explicitly. I replaced it by a helper that answers the question for any contact type and routes the sector case through the feature flag, turning a hard-coded enumeration into a single decision point.

The delivered feature therefore touches three places in the monorepo:

- **Administration panel** : the contact-type dropdown now offers the social worker of the member company, with an information box stating that the person will be resolved automatically.
- **Shared contact configuration** : a new category in the contact-section configuration package shared by the portals, which is what lets a service map this type onto its own team setup.
- **Employee portal** : the resolved social worker appears on the existing occupational-health card, next to the attachment centre and the referring physician.

One requirement changed late in the subject. My first implementation reused the shape of the referring physician's contact, and that shape includes a professional e-mail address. Product and compliance review concluded that a phone number is sufficient and that an individual mailbox does not belong on the employee portal, so I removed the e-mail from the payload, from the type and from the template. The delivered card carries the name and the contact phone number only.

VI.4 Phase 4: Validation and critical reflection

What deserves testing here is the resolution rather than the display, because the resolution depends on data that each service configures itself, sometimes in ways nobody anticipated. I covered it with integration tests on the query (a sector with one titular social worker, a sector with several, a sector with none, and a team member who is a social worker without being titular) and on the visibility rules of the contact section under each combination of the two feature flags. A further assertion checks that the employee-portal payload no longer carries an e-mail address, so that the compliance decision described above cannot be undone by accident.

Critical reflection : Resolving the contact instead of storing it removed the manual entry entirely, and that part of the design has held. Its weakness is the other side of the same choice: the card exists only for as long as the underlying assignment does. A sector left without a titular social worker publishes nothing, and the employee is given no explanation, only an absence. The single-row decision of Phase 2 treats the ambiguous case in the same spirit, preferring an imperfect answer to an empty one, but it addresses a symptom. What the feature still lacks is a fallback: when no titular professional can be resolved, the card should name the service's general support desk rather than disappear. I would also now question the lack of proactive validation on the administration side, where nothing warns a service, at the point of configuring this contact type, whether it currently resolves to anybody. The admin panel's contact preview tool can answer that question afterward, company by company, but only if someone thinks to check.

VII Case Study: Verifying and Enriching Automatically Created Prospects

What existed, and what I added : Part of the platform's growth is not commercial in origin. When a company hires an employee it must send a pre-employment declaration (DPAE) to the social security collection agency, and the declarations relevant to each occupational health service are retrieved according to criteria defined at the service level. The platform already ingested those files and already created a prospect record whenever the declaring company was not one it knew, so that the service could see that a company in its area had just hired somebody. What it did not do was check that this company exists and is still active, nor fill in what the declaration does not carry. Such a declaration holds little more than an identifier and an address, and in particular no telephone number, so the record arrived in the prospect list too thin to be acted upon. I added the verification and enrichment step that now runs before that record is created: a call to an external legal-information provider, a local cache so the same question is not paid for twice, and three distinct outcomes depending on what the provider answers.

The step is isolated behind a single feature flag, so that the new behaviour cannot reach production without a deliberate rollout.

VII.1 Phase 1: The existing system and its limits

The pipeline created a prospect from any declaration naming a company the platform did not already hold, and took the declaration at face value. Two consequences followed. First, nothing guaranteed that the company was real or still active: a mistyped identifier, or a company that had ceased trading since the declaration was filed, produced a prospect record like any other. Second, that record was poor. Since such a declaration carries no telephone number, the teams of the service, whose job it is to qualify these companies and get in touch with them, had to look each one up by hand, several hundred a day.

The product framing of this step was written as a set of acceptance scenarios rather than as prose. Read in order, they require that a declaration naming an establishment absent from the platform's own company tables triggers a verification of that company against an external source; that no prospect becomes visible, and no message is sent towards the company, on the strength of a declaration that has not been verified; and that no prospect is created at all if the verification finds no company for that identifier, or finds one that is no longer active. Those three scenarios are what the two skip paths of Phase 3 implement.

The second scenario deserves a word, because it describes a state the delivered pipeline never actually enters. It was written against a design in which the record would be created first and verified afterwards, which would leave a window during which an unverified prospect is visible. The implementation satisfies the requirement by construction instead: the verification runs before anything is written, in the same processing pass, so there is no intermediate state to hide and no outbound communication to hold back.

VII.2 Phase 2: Design decisions

The information missing from a declaration (legal status, activity, telephone number) is public, but it is not ours, so it is fetched from an external legal-information provider, queried on the establishment identifier carried by the declaration.

That provider bills per request, and asking for a telephone number costs several tokens, so calling it once per declaration was not acceptable: the same establishment appears in many declarations, and a company hiring ten people would generate ten identical questions. Two guards therefore sit in front of the call:

1. **Skip when the file already answers :** if the incoming declaration does carry a telephone number, no external lookup is made at all.
2. **Reuse what the provider already answered :** before any HTTP call, the engine looks the establishment up in a local table of previously fetched legal information. An entry still inside its validity window is reused as it stands. An entry past that window is marked invalid, a fresh call is made, and the new answer is written back with a new timestamp.

VII.3 Phase 3: What I implemented

I isolated that cache behind its own persistence repository, so that expiry and invalidation live in one place. I also replaced a legacy phone validator that accepted only the international E.164 format, and therefore dropped valid French domestic numbers, with a single expression that accepts both national and

international forms. If the value still fails, the phone is stored as empty and a warning is logged; the rest of the legal information is kept.

When the provider is called, three outcomes are distinguished:

- **Success** : The response is adapted into the internal legal-information shape. If the provider marks the company as no longer active, creation is skipped.
- **Not found (404 or identifier mismatch)** : If the provider answers that the establishment does not exist, or returns a different identifier from the one requested, creation is skipped entirely, so that an unverifiable company is never inserted.
- **Any other error (503, network, etc.)** : A temporary failure is logged, and the company is created from the raw declaration content without enrichment, so that an outage of the provider does not block ingestion.

VII.4 Phase 4: Validation and critical reflection

To validate the ingestion pipeline safely, I set up an asynchronous testing workflow:

- Created mock test archives containing diverse identifier and phone combinations.
- Uploaded these test archives into a cloud blob storage container.
- Executed the processing workflows manually via container command-line interfaces.
- Performed database state assertions by running raw SQL against the staging database from a shared notebook environment, inspecting the cached row and its validity timestamps.

Each run was launched as a workflow job on a dedicated peer-testing namespace and then audited through its streamed logs, where the outbound provider calls and the rejections described below are visible. The notebook environment was also used to force-expire a cached row on demand, which is how the cache-miss branch was exercised deliberately rather than by waiting for the validity window to lapse.

The execution states and outcomes observed across these runs are consolidated in the validation matrix of Table 1. Establishment identifiers are redacted, and the companies under test are designated by letter.

Target source	file	Company under test	Input state	Expected system outcome / assertions
Smoke archive		Company A	Valid phone in file	Record created immediately; external API call is bypassed.
Smoke archive		Company B	Missing phone in file	Provider answers with a different establishment; the identifier mismatch is raised as a 404-coded external API error and creation is skipped.
Smoke archive		Company C	Missing phone in file	External API provides the number successfully; record created with verified enriched data.
Validation archive		Fictional identifier	Missing phone in file	Provider returns HTTP 404; creation skipped entirely to prevent inserting an unverifiable company.
Integration suite		Company C	Missing provider stubbed on HTTP 503	Non-404 failure falls through the generic catch: a warning is logged and the company is created from the raw declaration content, without enrichment.
Validation archive		Company D	Missing phone in file	Active cache lookup match found; number retrieved locally from the legal-information table with zero external API fees.

Table 1: Asynchronous test execution matrix and pipeline validation states. All rows but one correspond to archives processed on the peer-testing cluster; the provider-outage row is a failure injected in the integration suite, since a transient outage cannot be reproduced on demand against the live API

Critical reflection : The caching architecture successfully protected the API token budget, and the guard that skips the lookup entirely when the file already carries a phone number turned out to remove more calls than the cache itself. Error handling was deliberately split between critical data mismatches and network errors: if the external API returns a 404 or an identifier mismatch, the system safely skips company creation entirely to prevent invalid database records. In contrast, if the API experiences a temporary non-404 outage, the engine logs a warning and falls back to creating the company record directly from the

declaration content without enrichment, balancing system availability against data completeness.

What I would revisit is the silence of the fallback path. A company created without enrichment because the provider was down is indistinguishable, afterwards, from one created before the feature existed: nothing marks it as awaiting a retry. A flag on the record, and a periodic pass over the records carrying it, would close that gap at a very modest cost, and its absence is a design omission rather than a deliberate trade-off.

VIII Case Study: Bulk Revocation of a User's Access Across Every Member Company

This case study describes a feature that lets an occupational health service revoke, in a single action, every platform access of a person who administers several member companies. It is the largest of the four subjects, cut into fourteen development tickets and delivered incrementally over several sprints.

What existed, and what I added : A member company is administered on the employer portal by one or more *employer users*, and the data model holds one access row per person and per company. The platform could already revoke one of those accesses: an operator opened the file of a company, found the person there, and revoked the access from that file. That single-company path also enforced the two rules that make a revocation safe, refusing to remove the last administrator of a company and handing over the notifications for which the departing person was the only recipient. What did not exist was a way to do this across all the companies of one person at once. I built that path: two endpoints, one that previews the operation and one that queues it, a background worker performing the revocations in chunks, and the dialog through which an operator chooses the scope and sees what the operation is about to do.

As in the previous cases the whole feature is isolated behind a feature flag, set per client, so that neither the interface nor the endpoints exist for a service that has not been deliberately opted in.

VIII.1 Phase 1: The existing system and its limits

Revoking the accesses of a person who administers twenty companies meant opening twenty company files and repeating the same operation twenty times. In practice the operation did not stay inside the interface at all. The action most frequently requested from customer support was exactly this one: revoke all of a person's accesses at once. Support forwarded the request to the development team, which executed it as a hand-written query run from the shared notebook environment already described in the previous case study.

That detour carried two costs, and the product framing names them both. The first is time, spread across the user who waits, the customer care team that relays and the developers who are interrupted. The second, and the more serious one, is correctness. The two safety rules recalled above are enforced in the application code of the single-company revocation, not by a constraint inside the database, so a query written by hand runs underneath them: it can leave a company with no administrator, or leave a notification whose only recipient has just been removed, precisely because it bypasses the code that would have refused to.

One detail of that framing is worth noting because it is easy to misread as a contradiction: although this was the most frequently requested support action, its deadline field read *when we have time, not a priority*. Each individual occurrence was cheap enough to absorb by hand; only the accumulation was expensive, and accumulation is not what a prioritization board tends to see.

VIII.2 Phase 2: Design decisions

Four decisions shaped the implementation.

Running the work in the background. Since a person may administer many companies, the request only validates the demand and queues an asynchronous task, that is a unit of work picked up later by a background process, cut into chunks of at most one hundred accesses on the pool reserved for long operations.

Reusing, or not reusing, the existing single-company revocation. This was the real arbitration, written down and defended during the analysis ritual. Calling the per-company routine once per company would keep the business rules in one place but repeat its most expensive step, the recalculation of notifications, for every company. A dedicated routine treating the batch as one set of rows means far fewer database round trips and two code paths to keep aligned. The team chose the second, accepting a known consequence: each behaviour of the per-company path had to be re-established on the new one, becoming its own ticket scheduled after the job it depends on.

Deciding what to do about the last remaining administrator. Revoking the only administrator left in a company leaves it unmanaged. Rather than hard-coding one behaviour, the feature reads a second,

pre-existing feature flag resolved for the individual requesting user rather than for the client: without it those companies are excluded from the batch, with it they are kept and reported to the operator.

Refusing to revoke a set the operator never saw. The interface asks the server how many companies would be affected, displays that number, and returns it with the confirmation. The server recomputes the set and rejects the request if the two disagree, guarding against another operator or an automated import changing the accesses in between.

VIII.3 Phase 3: What I implemented

On the server side, the entry point removes duplicate identifiers, asks the database which of the requested accesses are still revocable, and revokes that subset only, each chunk running inside one transaction. The version first delivered instead demanded that every identifier be revocable and refused the whole chunk otherwise; Phase 4 explains why that had to be relaxed. Before writing, the batch hands over the notifications the dialog announces: where the revoked person is the only recipient of a notification of their company, it passes to the remaining administrators, reusing unchanged the routine the per-company path already called.

Revocation appends to history rather than erasing: the open record is closed by writing a date of end, and a new record carrying the revoked status and the identity of the operator is inserted beside it, the same convention as the date-of-end field of the first case study. Nothing is deleted. The consequence is that "is this access active today" has to be answered by a rule rather than by reading one column, so that rule is written once as a reusable SQL fragment: an access counts as active either when a granted record still has an empty date of end, or when no history record exists at all, the second branch covering the rows that predate the history table. A second fragment builds on the first to express that a row carries the administrator role while no other row of the same company carries it and is active by that same definition. A single conditional then decides whether the sole-administrator clause is appended or omitted, which is how one statement serves both the strict policy and the permissive one.

Two endpoints sit on top, one returning the counts the dialog shows and one queueing the task, while the listing behind each count extends an existing route with an optional parameter rather than adding a new one.

The dialog offers two scopes, the company currently open by default or every company, and derives each warning it displays from the counts the server returned rather than from anything the operator typed. The lists behind those counts are rendered by a single presentational component I extracted for the purpose, now reused in five places. None of the wording lives in the component: every sentence is resolved against two axes, the language and the type of customer, so that the same screen names the administered entity as a member company for a mutualized service and as an operational unit for an autonomous one. One codebase with a vocabulary overlay, not a second version of the product.

Completion is not awaited either. A shared service already rendered progress and completion messages for any background task it was told to watch, pushing them over the WebSocket connection the application keeps open anyway, so this feature had only to declare the task type and the user the result is meant for. My first version of the dialog had followed its own task instead, by asking the server for a status every two seconds; registering the task with that shared service replaced the loop, and the only server-side change it required was one added line naming the user the push is meant for.

VIII.4 Phase 4: Validation and critical reflection

The feature was validated at three levels rather than through manual exercise alone, because the interesting cases are combinations of two feature flags and two scopes, which are tedious and unreliable to reproduce by hand. The resulting coverage is summarized in Table 2.

Level	Scenario under test	Expected system outcome / assertions
Worker (background)	Batch mixing active accesses and legacy rows with no history record	All are revoked; the legacy rows are treated as active and are not skipped.
Worker (background)	Batch containing identifiers that are no longer revocable, or a person deleted in the meantime	Those identifiers are skipped and the remaining accesses are still revoked, so one stale entry does not cancel the batch.
Worker (notifications)	Revoked person is the only recipient of a notification of their company	The notification is handed over to the remaining administrators of that company, while a notification that already has other recipients is left untouched.
Worker (notifications)	Batch mixing revocable and non-revocable identifiers, notifications attached to both	The hand-over runs for the revocable ones only, which is the same subset the revocation itself applies to.
API (preview)	Person administering several companies, sole administrator of some of them	The two counts are returned separately; sole-administrator companies are removed from the concerned count and reported in the second one.
API (confirmation)	Displayed count no longer matches the real set	Request refused with an explicit error rather than revoking a different set from the one shown.
API (confirmation)	Person with no active access left	Request refused, so an empty task is never queued.
API (both flags)	A requester holding the last-administrator flag, and a requester without it, on the same data	Sole-administrator companies are included for the first and still excluded for the second, confirming the flag is resolved per user and not per client.
API (feature flag off)	Both endpoints called without the protecting feature flag	Both are refused, confirming the feature is fully inert for a service that has not opted in.
Interface	The four combinations of scope and last-administrator flag	The expected warning is displayed in each, and the all-companies option is disabled outright when the flag is off and only one company is concerned, so an operator cannot submit a request the server would reject.
Interface	Server answers with the count mismatch error	The message is shown inline and the count is refetched immediately, so a retry is made against fresh numbers.

Table 2: Validation matrix of the bulk revocation feature across the background worker, the two endpoints and the interface. The flag combinations are covered by automated tests rather than manual runs, since reproducing them by hand on a shared environment is unreliable

Critical reflection : The arbitration of Phase 2 had a price, and what makes this case study instructive is that one half of that price was anticipated and the other half was a defect of my own making.

The anticipated half is the direct consequence of the arbitration. Writing a dedicated batch routine rather than calling the per-company one in a loop made the batch fast, and in exchange every behaviour of the per-company path had to be re-established explicitly on the new one. The team knew this when it chose, and the slicing shows it: the hand-over of notifications, by which a notification whose only recipient is being revoked passes to the remaining administrators of that company, was cut as its own ticket from the start, scheduled after the background job it depends on. The interface was built first and its wording

written for the finished behaviour, which is why the dialog already announced the hand-over before the hand-over existed. That ordering is only safe because of the feature flag: the whole feature stays inert for any service that has not opted in, so an interface slice and a behaviour slice can land in different sprints without any client ever seeing the intermediate state. Working this way is what allows a user story of fourteen tickets to be delivered continuously instead of in one large drop, and the copy never had to be rewritten because it was written once, for the end state.

The other half was not anticipated, because it was a defect I introduced, and naming it plainly is the whole point of mentioning it. While implementing the hand-over I let one rule be written twice in two different languages. The question "which of these accesses can still be revoked" was answered on the endpoint by a raw SQL fragment, and inside the background job by a count issued through the object-relational layer. Each looked correct on its own, and the two disagreed on the awkward rows, in particular the accesses that carry no history record at all because they predate that table. The strict guard described in Phase 3 then did exactly what it had been written to do, and refused entire batches for reasons that were entirely benign. The correction came in two parts: the rule now lives in one shared module that both the hand-written queries and the generated ones consume, and the job skips what it can no longer revoke instead of refusing everything. The lesson is narrower than "avoid duplication" and I prefer to keep it in its narrow form: a rule is written once only if every caller actually goes through it, and a fragment used by three queries out of four is not a single source of truth, it is a convincing imitation of one.

The refactoring that reunifies the per-company and batch paths, so that the divergence stops having to be managed by hand, has since been written and reviewed. What remains open is a question of performance: the hand-over asks for the notification statistics once per company in the batch, which is tolerable for chunks of one hundred but was accepted during weighting without ever being measured, and became a ticket of its own once review made the cost concrete. Recording it rather than leaving it implicit is, I think, the honest outcome of the arbitration rather than a failure of it.

IX Conclusion and Personal Assessment

IX.1 Professional Summary & Collaborative Outcomes

Working within a sub-squad owning the lifecycle of member companies and prospects allowed me to participate in a highly collaborative engineering culture. By contributing to key modules such as duplicate tracking, the social worker contact resolved on both portals, the declaration ingestion pipeline and the bulk revocation of member-company accesses, I gained a comprehensive view of a large product's software architecture. Beyond my individual case studies, I participated in code quality and team velocity initiatives:

- Reviewing code changes through daily pull request peer reviews, ensuring consistency with the team's coding guidelines.
- Collaborating with Product Managers and designers during design alignment meetings to prevent design flaws before implementation.
- Participating in collective testing sessions to validate features across different user environments and edge cases.

IX.2 Personal Development & Engineering Posture

My end-of-studies internship has been a vital phase in my transition from an academic student at Ecole Centrale de Nantes to an autonomous software engineer. On the technical side, working in a large-scale, event-driven monorepo improved my skills in frontend reactive programming, asynchronous data pipelines and search index optimization; it also taught me a category of problem that academic projects rarely expose, namely eventual consistency between two persistence layers that are never updated in the same transaction.

The most significant shift, however, concerns my professional posture, and it happened in three steps.

- **From executing tickets to framing them :** during my first weeks I implemented what the ticket described. The analysis and weighting ritual forced the opposite habit: presenting a written technical plan, defending it in front of the other members of the sub-squad, and accepting that the reviewed solution is frequently not the one I arrived with. Being challenged on the search-index pre-calculation strategy before writing a line of code saved a rewrite, and taught me that the cheapest place to be wrong is the whiteboard.
- **From avoiding mistakes to owning them :** the event-bus synchronization defect found during collective testing was, objectively, a gap in my own analysis. Reporting it explicitly to the team, tracing the root cause instead of patching the symptom, and writing the regression tests that prevent its return, was more professionally formative than any feature I shipped. I now consider a fix incomplete until the test that would have caught it exists.
- **From individual output to team throughput :** the rotating Scrum Master and on-call roles moved my attention from my own tickets to the sub-squad's flow: unblocking colleagues, arbitrating between an urgent production incident and the sprint commitment, and communicating with Product Managers and designers in their own vocabulary rather than in technical terms.

Two areas remain explicit development axes for me. The first is estimation: I consistently under-weighted tickets during the first sprints, mostly by underestimating the cost of cross-team coordination on shared packages. The second is knowing when to stop investigating alone; I learned late that asking my buddy after thirty minutes of being stuck is not a failure of autonomy but a correct use of the team's time.

Overall, this internship confirmed my wish to work as a full-stack engineer on products with a tangible societal purpose, and gave me the confidence that I can take responsibility for a feature end-to-end, from its functional framing to its production monitoring, inside a demanding engineering organization.

References

- [1] INRS (Institut National de Recherche et de Sécurité), *Organisation des services de prévention et de santé au travail*, INRS Publications, Technical Report, Reference: ED-6485, Paris, updated 2023. Available at <https://www.inrs.fr/demarche/services-sante-travail/organisation.html>.
- [2] Arnaud Mias, *L'horizon fuyant de la santé au travail*, SciencesPo LIEPP Policy Brief, n° 64, LIEPP Publications, Paris, May 2024. Available at <https://www.sciencespo.fr/liepp/fr/actualites/arnaud-mias-l-horizon-fuyant-de-la-sante-au-travail/>.
- [3] ADSP (Actualité et dossier en santé publique), *Santé et travail — Les origines, les acteurs et les indicateurs*, ADSP Publications, Paris, ISSN: 1243-115X, n° 9, December 1994.
- [4] Claire Touaillat, *Quels sont les points clés du rapport 2022 sur l'activité des services de prévention et de santé au travail ?*, Lefebvre Dalloz Publications, Paris, ISBN: 978-2-247-21650-6, July 2022.

X Glossary

This glossary defines the main industry, regulatory and technical terms used throughout the report, so that it can be read by someone outside the project. French regulatory acronyms are kept in their original form, since they have no official English equivalent.

Member company (adhérent) :

Company that has joined an occupational health service, which in exchange monitors its employees. *Adhérent* is the term used in the product interface; this report says member company throughout.

Prospect :

Company that has not joined a service yet, either identified and created by the service itself (commercially, or automatically from a pre-employment declaration) or created by the company itself through an eligibility questionnaire. Prospects and member companies live in separate tables, which is what makes the duplicate detection of Section V necessary.

DPAE (Déclaration Préalable à l’Embauche) :

Mandatory declaration an employer must send to the social security collection agency before hiring any employee. It carries the administrative data used to create prospect records automatically.

SIREN (Système d’Identification du Répertoire des Entreprises) :

Unique 9-digit number assigned by INSEE to every French company at registration. It is the pivot identifier used for the duplicate detection of Section V.

SIRET (Système d’Identification du Répertoire des Établissements) :

14-digit number identifying each physical establishment of a company, formed by the 9-digit SIREN followed by a 5-digit establishment number.

SPSTI (Service de Prévention et de Santé au Travail Inter-entreprises) :

Non-profit body grouping several companies together to pool the medical monitoring and prevention activities covering their employees.

SPSTA (Service de Prévention et de Santé au Travail Autonome) :

Internal occupational health service belonging to a single large company, handling prevention and medical follow-up exclusively for its own employees.

HDS (Hébergeur de Données de Santé) :

Mandatory French certification for any entity hosting or processing personal health data, guaranteeing a high level of security and confidentiality.

Asynchronous task (worker) :

Unit of work queued by the application and executed later by a background process, so that a long operation does not block the request that triggered it. A large batch is cut into several jobs belonging to the same task, which is how the bulk revocation of Section VIII is executed.

Feature flag :

Development technique that enables or disables a feature in production without deploying new code. The squads use it to isolate work in progress and to open a feature client by client.

Collective testing : Peer-testing ritual in which the whole sub-squad exercises a feature together across several runtime environments before release.

On-call rotation : Weekly rotation in which one engineer steps out of sprint work to handle production incidents and support tickets.